

DocuGenAI

Application Owner: EY Advisory S.p.A.

Document Version: 1.0

Reviewers: Tiziano Bardini, Alessio Buda, Emanuela Rremilli, Danilo Santo

Key Links

- [GitHub](#)

General Information

Regulatory References: [Article 11](#); [Annex IV](#) paragraph 1, 2, 3

Purpose and Intended Use

- **Description:** This AI system is a CrewAI-based report generation tool designed specifically for consulting firms operating in the software development and technology advisory sector. The system automates the creation of comprehensive project reports by analyzing project documentation and generating structured technical summaries, evaluations, and critical analyses.
- **Problem Statement:** The system addresses the time-intensive challenge faced by developers and consultants in producing final project reports. Currently, creating comprehensive technical architecture summaries, project evaluations, execution examples, and critical analysis of results requires significant manual effort and time investment. This AI-powered tool streamlines this process by providing an intelligent first draft based on uploaded project documentation and developer-defined outlines.
- **Target users and stakeholders:**
 - **Primary users:** Software developers and technical consultants working in consulting firms
 - **Secondary stakeholders:** Project managers, client-facing teams, and end clients who receive the generated reports
 - **Organizational stakeholders:** Consulting firm leadership seeking to improve operational efficiency and service delivery
- **Measurable goals and key performance indicators (KPIs):**
 - **Primary KPI:** Reduction in time required for report generation (target: 60-80% time savings compared to manual report creation)
 - **Quality metrics:** User satisfaction scores for report accuracy and completeness
 - **Efficiency metrics:** Number of reports generated per hour/day
 - **Adoption metrics:** User engagement and system utilization rates within consulting teams
- **Ethical implications and regulatory constraints:** Given the system's focus on technical documentation analysis without processing personal data or making decisions that significantly impact individuals, ethical risks are minimal. The system operates within standard business process automation

boundaries and does not involve sensitive personal information or high-stakes decision-making scenarios.

- **Prohibited uses and potential misuse scenarios:** The system must not be used to generate reports for projects outside the user's legitimate access and responsibility. Reports should not be presented as final deliverables without appropriate human review and validation. The system should not be used to create misleading or false technical documentation. Generated content must be clearly identified as AI-assisted when shared with clients or external stakeholders.
- **Operational environment:** The AI system operates as a command-line interface (CLI) tool deployed on Azure cloud infrastructure, utilizing Azure AI Foundry models (GPT-4o for text generation and text-embedding-ada for document analysis). Users interact with the system by uploading project documentation to a designated folder structure and providing report outlines. The system processes this information in real-time to generate comprehensive technical reports. The operational environment is designed for secure, on-demand report generation within consulting firm internal networks and approved cloud environments.

Risk Classification

Regulatory References: [Article 5](#)

Classification: Limited Risk (in accordance with the AI Act [Article 50](#))

Reasoning: This AI system is classified as Limited Risk under Article 50 of the EU AI Act based on the following assessment:

- The system directly interacts with natural persons (clients and stakeholders) through the generated reports that are shared as part of consulting deliverables.
- The AI-generated content could influence business recommendations and - project assessments, creating a potential impact on decision-making processes. The system generates content that is presented to end users (clients) as part of professional consulting services.
- Transparency obligations are necessary and implemented, as clients are informed about AI assistance in report generation.
- The system does not fall under any of the specific high-risk categories listed in Annex III of the AI Act.

Risk Mitigation Factors:

- Mandatory human review and approval process before report finalization. Reports are based on existing project documentation rather than autonomous analysis.
- Users retain full editorial control and responsibility for final deliverables.
- Clear transparency requirements ensure clients are aware of AI involvement. The system serves as an assistive tool rather than a replacement for human expertise and judgment.

Application Functionality

Regulatory References: [Article 11](#); [Annex IV](#), paragraph 1, 2, 3

Instructions for Use for Deployers

(EU AI Act [Article 13](#))

Model Capabilities

What the application can and cannot do:

Can do:

- Generate comprehensive technical reports based on project documentation
- Process and analyze Sphinx-generated HTML documentation, Markdown files, and PDF documents
- Create customized report outlines based on user-defined specifications and target audience
- Perform intelligent document search and retrieval using RAG (Retrieval-Augmented Generation) technology
- Adapt content complexity based on specified audience (technical vs. non-technical)
- Sanitize and validate input parameters to ensure system stability

Cannot do:

- Generate reports without adequate source documentation
- Create content beyond the scope of provided project materials
- Process proprietary file formats outside of supported types
- Guarantee 100% accuracy without human review and validation
- Operate effectively with incomplete or poorly structured documentation

Supported languages, data types, or scenarios

Languages: Optimized for English

Data types: Markdown (.md), PDF (.pdf), HTML (.html) files

Optimal scenario: Complete Sphinx-generated documentation with comprehensive project coverage

Supported scenarios: Technical architecture documentation, project evaluation reports, implementation summaries

Input Data Requirements

Format and quality expectations:

- **Required folder structure:** project folder > src > "project_name" > tools > docs
- **File placement:** All documentation must be located in the designated docs directory
- **Supported formats:** .md, .pdf, .html files
- **Quality expectations:** Well-structured documentation with clear hierarchical organization (Sphinx-generated documentation preferred)

Input parameters

- Project description
- Expected report outline structure
- Target audience specification (technical/non-technical)

Output Explanation Report format and structure:

- **Output format:** Markdown (.md) files

- **Structure:** User-defined outline with populated sections based on documentation analysis
- **Content adaptation:** Technical depth adjusted based on specified target audience

Interpretation guidelines

Generated reports represent AI-assisted analysis of provided documentation Content accuracy depends on the quality and completeness of input documentation All outputs require human review and validation before client delivery Reports serve as first drafts requiring professional oversight and potential modification

System Architecture Overview

Functional description: The system implements a three-crew CrewAI architecture with specialized agent roles for document processing and report generation. **Key components and workflow:**

- 1. Crew 1 - Input Processing:
 - Agent 1: Input validation, including safety evaluation
 - Agent 2: Input sanitization and parameter cleaning
- 2. Crew 2 - Content Planning:
 - Agent 1: Project detail extraction and audience determination
 - Agent 2: Dynamic outline creation based on user specifications and content analysis
- 3. Crew 3 - Report Generation:
 - Agent 1: RAG-powered documentation searcher with Qdrant vector database integration
 - Agent 2: Report writer that synthesizes outline structure with retrieved documentation content

Technical infrastructure:

LLM Integration: Azure OpenAI API with GPT-4o and text-embedding-ada models

Vector Database: Qdrant for document indexing and semantic search Document Processing: Multi-format parser supporting HTML, Markdown, and PDF Deployment Environment: Azure AI Foundry infrastructure Interface: Command-line interface (CLI) for user interaction

Data flow:

User provides project description, outline, and audience specification via CLI Input validation and sanitization crew processes parameters Content planning crew analyzes requirements and expands outline Report generation crew performs RAG search and populates outline with relevant documentation content System outputs structured Markdown report for human review

Models and Datasets

Regulatory References: [Article 11](#); [Annex IV](#) paragraph 2 (d)

Models

Link to all model integrated in the AI/ML System

Model	Link	Description of Application Usage
-------	------	----------------------------------

Model	Link	Description of Application Usage
GPT-4o	Azure AI Foundry Endpoint	Primary language model used for text generation and tool use across all six agents in the three-crew architecture. Handles input sanitization, project analysis, outline creation, documentation search, and report writing tasks.
text-embedding-ada-002	Azure AI Foundry Endpoint	Text embedding model used for document vectorization and semantic search capabilities. Converts project documentation into vector representations stored in Qdrant database for RAG-based information retrieval during report generation.

Datasets

No datasets other than the documentation provided by the user are employed in the project.

Deployment

Infrastructure and environment details:

Environment: Local development/testing environment

Platform: Personal computer running CLI application

Cloud services: Azure AI Foundry for model access via API endpoints (<https://ai-academy-buda.services.ai.azure.com/>)

Vector database: Qdrant instance (local or cloud-hosted for document vectorization and retrieval)

Authentication: Azure OpenAI API key authentication

Runtime: Python environment with CrewAI framework dependencies

Integration with external systems or applications

Regulatory References: [Article 11](#); [Annex IV](#) paragraph 1 (b, c, d, g, h), 2 (a)

Azure OpenAI Service: API integration for GPT-4o and text-embedding-ada-002 models

Qdrant Vector Database: For document embedding storage and semantic search capabilities

APIs

The project is not currently available via API.

Infrastructure

Since the project is currently under testing, no specific infrastructure has been deployed yet.

Lifecycle Management

Regulatory References: [Article 11](#); [Annex IV](#) paragraph 6

The project is currently in its first stages of testing and no lifecycle management has been defined.

Metrics

- **Application performance:** not currently evaluated
- **Model performance:** each crew is individually evaluated. The first and second crew are tested for their ability to sanitize the input and provide a coherent outline respectively. These aspects are evaluated using a LLM-as-a-judge method. The RAG tool of the last crew has been individually evaluated using RAGAS. Results may be found in an attached document.
- **Infrastructure:** not currently evaluated

Key Activities

- Monitor performance in real-world usage.
- Identify and fix drifts, bugs, or failures.
- Update the model periodically.

Documentation Needs

Since the project is not currently deployed, no documentation needs apply.

Risk Management System

Regulatory References: [Article 9](#), EU AI Act [Article 11](#); [Annex IV](#)

Risk Assessment Methodology

Practical risk identification based on system functionality and testing observations. Risks are assessed through direct testing and evaluation of system outputs in controlled scenarios.

Identified Risks

Potential Harmful Outcomes:

- **Inaccurate technical analysis:** AI may misinterpret documentation or generate incorrect technical assessments
- **Incomplete report generation:** System may miss important project details due to documentation gaps or processing limitations
- **Over-reliance on AI output:** Users may accept AI-generated content without adequate review
- **Misleading project representation:** Reports may inadvertently misrepresent project capabilities or limitations

Likelihood and Severity:

- **Inaccurate analysis:** Medium likelihood, Medium severity (mitigated by human review requirement)
- **Incomplete reports:** Low likelihood, Low severity (evident during review process)
- **Over-reliance:** Low likelihood, Medium severity (addressed through user training and transparency)
- **Misleading representation:** Low likelihood, Medium severity (controlled through mandatory review process)

Risk Mitigation Measures

Preventive Measures:

- Input validation and sanitization through dedicated agents
- RAG-based approach ensures responses are grounded in provided documentation
- Structured three-crew architecture with built-in quality checks
- Clear transparency requirements about AI involvement in report generation

Protective Measures:

- Mandatory human review and approval before any report distribution
- Clear labeling of AI-assisted content
- Testing phase allows for functionality validation and improvement
- Users maintain full editorial control over final deliverables
- System operates as assistive tool rather than autonomous decision-maker

Testing and Validation (Accuracy, Robustness, Cybersecurity)

Regulatory References: [Article 15](#)

Testing and Validation Procedures (Accuracy)

- **Performance Metrics:** For the sanitize crew and analysis crew, an LLM evaluates the success rate of the crew. For the RAG tool, context precision, context recall, faithfulness, and answer relevancy are evaluated on a set of input - ground-truth pairs
- **Validation Results:** Sanitize and analysis crew: ADD RESULTS. RAG Tool:
 - context_precision 0.808333/1
 - context_recall 0.788889/1
 - faithfulness 0.954412/1
 - answer_relevancy 0.912678/1

Accuracy Throughout the Lifecycle

Data Quality and Management

Users should provide high-quality documentation for optimal results.

Model Selection and Optimisation

Models, such as GPT-4o and text-embedding-ada-002, have been select since they are well-known and provide high-quality results. Additional model coyuld be tested in the future.

Feedback Mechanisms

- **Real-Time Error Tracking**
- N/A

Robustness

The system has not been currently evaluated for rubustness

Cybersecurity

Regulatory References: [Article 11](#); [Annex IV](#) paragraph 2 (h)

Data Security

- **Current state:** Local testing environment with project documentation processed locally
- **API security:** Azure OpenAI API authentication via secure API keys
- **Data handling:** No persistent storage of sensitive data; documentation processed temporarily during report generation
- **Vector database:** Qdrant stores document embeddings without sensitive personal information

Access Control

- **Current state:** Single-user testing environment on local machine
- **API access:** Restricted to configured Azure OpenAI endpoints with authenticated access
- **File access:** Limited to designated documentation folders with standard file system permissions

Incident Response

- **Current approach:** Manual monitoring during testing phase
- **Error handling:** System logs processing errors and failures for debugging
- **Recovery:** Manual restart and re-processing capabilities available
- **Escalation:** Direct developer oversight during testing phase

Testing Phase Considerations:

Given the current local testing environment, cybersecurity risks are minimal as the system operates in a controlled, single-user environment without network exposure or sensitive data processing. Standard security practices include secure API key management and local file system security.

Incident Management

Azure API Connection Issues:

- **Issue:** Authentication failures or API timeouts Solution: Verify API keys and endpoint configuration, check internet connectivity
- **Debug:** Review API response codes and error messages

Qdrant Database Connection Problems:

- **Issue:** Vector database unavailable or connection errors Solution: Restart Qdrant service, verify connection parameters
- **Debug:** Check Qdrant logs and network connectivity

Support Contact

- **Primary contact:** Project developer (testing phase)
- **Technical issues:** Review system logs and CLI error outputs
- **Documentation:** Refer to CrewAI and Azure OpenAI service documentation

EU Declaration of Conformity

Regulatory References: [Article 47](#)

- System Name: TO BE DEFINED
- Provider: EY Advisory S.p.A. Via Meravigli, 12/14, 20123 Milano MI
- Compliance: EU AI Act Art. 47, GDPR, ISO/IEC 27001

Standards Applied

N/A

Documentation Authors

- Tiziano Bardini: Contributor
- Alessio Buda: Contributor
- Emanuela Rremilli: Contributor
- Danilo Santo: Contributor